LEIC - Organização de Computadores (2026/2027)

Laboratory Assignment 1: System Profiling and Modeling

Section 2: Exploring Cache Behaviour Through Timing

Question a) Experimental Environment

- **Processor model:** Intel(R) Xeon(R) CPU E5-2650 0 @ 2.00GHz
- **Operating system:** NixOS 25.05 (Warbler)
- **Compiler version:** GCC 14.2.1
- **Compilation options:** `-O2 -std=gnu11 -Wall -Wextra`

Question b) 16 KiB Array Access Pattern

Stride (bytes)	First four byte offsets accessed	Number of elements updated per pass
1	0, 1, 2, 3	16,384
32	0, 32, 64, 96	512
64	0, 64, 128, 192	256

Explanation: The calculations above are based on an array size of 16,384 bytes (16 KiB). The first four byte offsets follow the sequence 0, s, $2s$, and $3s$, where s is the access stride. The total number of elements updated per pass is determined by dividing the total array size by the stride (N/s), demonstrating that as the stride increases, fewer memory locations are accessed within a single pass across the working set.

Question c) L1 Data-Cache Capacity Estimate

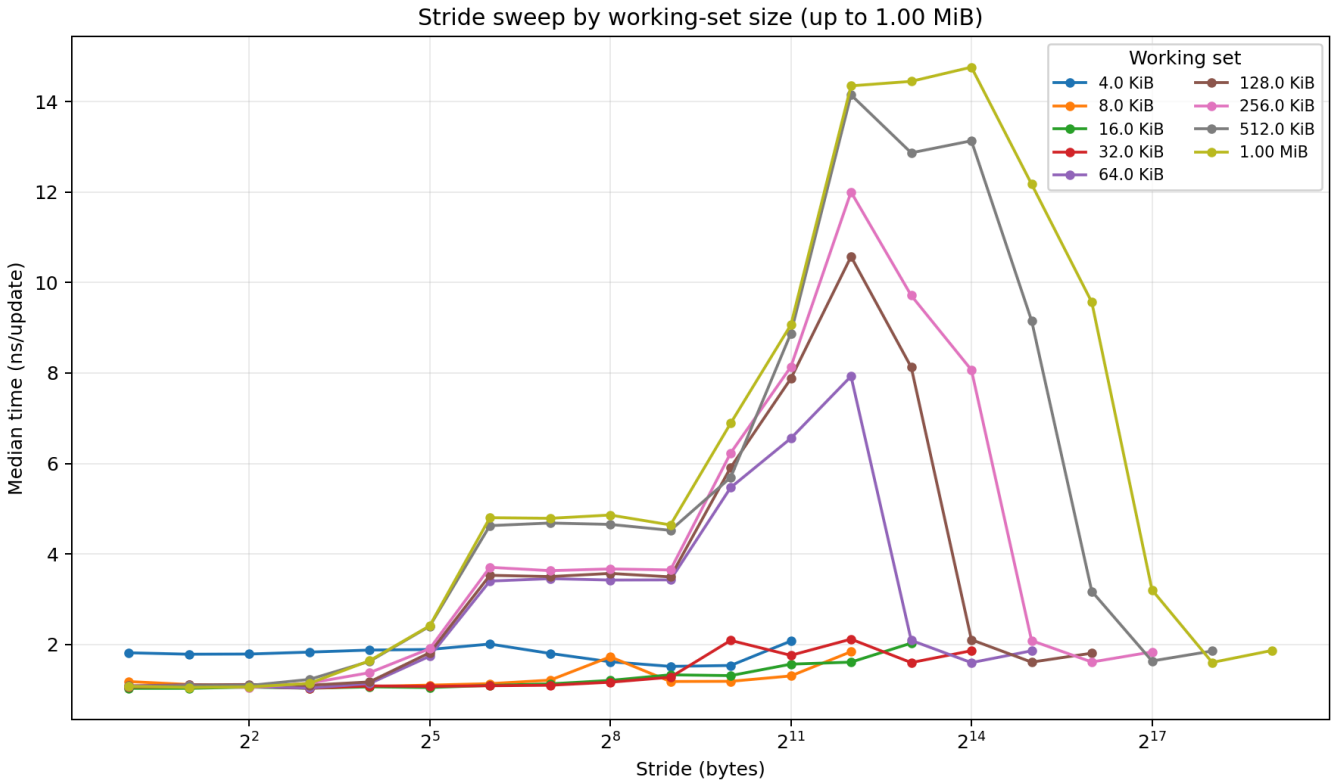


Figure 1: Median time per update (ns) vs. stride (bytes) across working set sizes up to 1.00 MiB.

- **Estimate:** 32 KiB
- **Justification:** By analyzing the plot and the summary table, arrays up to 32 KiB maintain a consistently low execution time (e.g., 1.09 ns/update at a 64-byte stride). When the working set increases to 64 KiB, there is a distinct, sustained upward shift in the execution time (jumping to 3.40 ns/update at the same 64-byte stride). This sharp increase indicates that the 64 KiB array no longer fits entirely within the L1 data cache, resulting in capacity misses and forcing the CPU to retrieve data from the slower L2 cache.

Question d) L2 Data-Cache Capacity Sweep

- **Minimum array size used:** 64 KiB
- **Maximum array size used:** 64 MiB

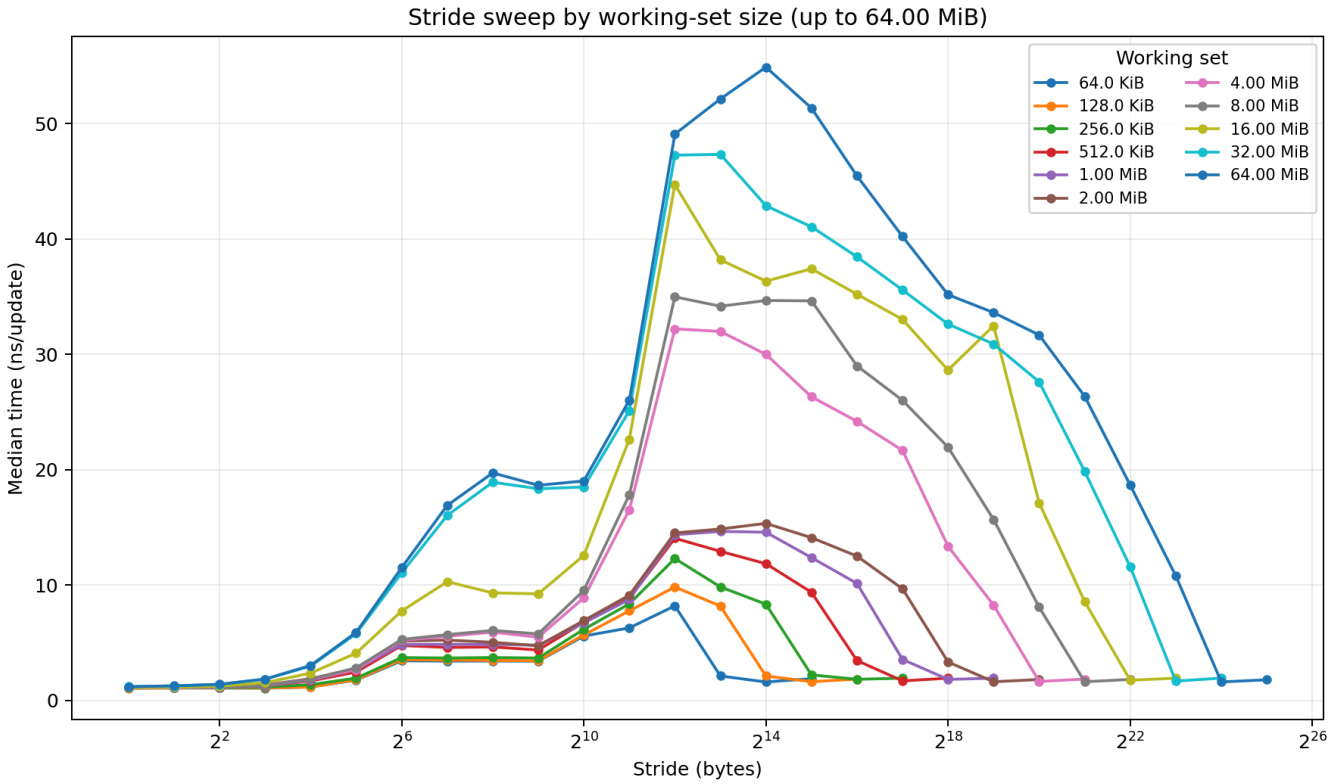


Figure 2: Stride sweep by working-set size for L2 investigation (64 KiB to 64 MiB).

- **Estimated L2 Capacity:** 256 KiB (or interval [256 KiB, 512 KiB])

Justification:
Comparing the timing curves across working set sizes at identical strides reveals that working sets of 64 KiB, 128 KiB, and 256 KiB exhibit almost identical, low execution times (e.g., ~3.43 ns to 3.71 ns at a 64 B stride). When the array size increases from 256 KiB to 512 KiB, a clear, sustained jump in execution time occurs across all measured strides (e.g., increasing from 3.71 ns to 4.76 ns at 64 B stride, and from 12.30 ns to 14.06 ns at 4 KiB stride). This sustained rise indicates that array sizes exceeding 256 KiB can no longer fit within the L2 data cache and must be retrieved from the slower L3 cache or main memory. This 256 KiB estimate aligns with the per-core L2 cache hardware specification of the Intel Xeon E5-2650 processor.

Question e) L1 Data-Cache Line Size Estimate

- **Selected Array Size:** 128 KiB (Larger than the 32 KiB L1 capacity, smaller than the 256 KiB L2 capacity).
- **Estimated L1 Cache Line Size:** 64 bytes.

Justification:
As the stride increases, spatial reuse within a fetched cache line decreases, causing the time per update to rise. Once the stride is equal to or greater than the cache line size, every update requires fetching a new line, causing the execution time to plateau.

Looking at the 128 KiB working set, the median time per update rises from 1.17 ns at a 16 B stride, to 1.82 ns at a 32 B stride, and reaches 3.53 ns at a 64 B stride. For strides of 128 B, 256 B, and 512 B, the time plateaus at approximately 3.50 ns. This identical plateau pattern at 64 B is also visible on the 64 KiB and 256 KiB working sets. Because the time per update stops rising at a stride of 64 bytes, it indicates that spatial reuse has been fully exhausted, confirming a cache line size of 64 bytes.

Section 3: Exploring Cache Behaviour Through Hardware Counters

Question a) Experimental Environment & Event Details

- **Processor model:** Intel(R) Xeon(R) CPU E5-2650 0 @ 2.00GHz
- **Operating system:** NixOS 25.05 (Warbler) / Linux 6.12.40
- **Compiler version:** GCC 14.2.1
- **Compilation options:** `-O2 -std=c11 -Wall -Wextra`

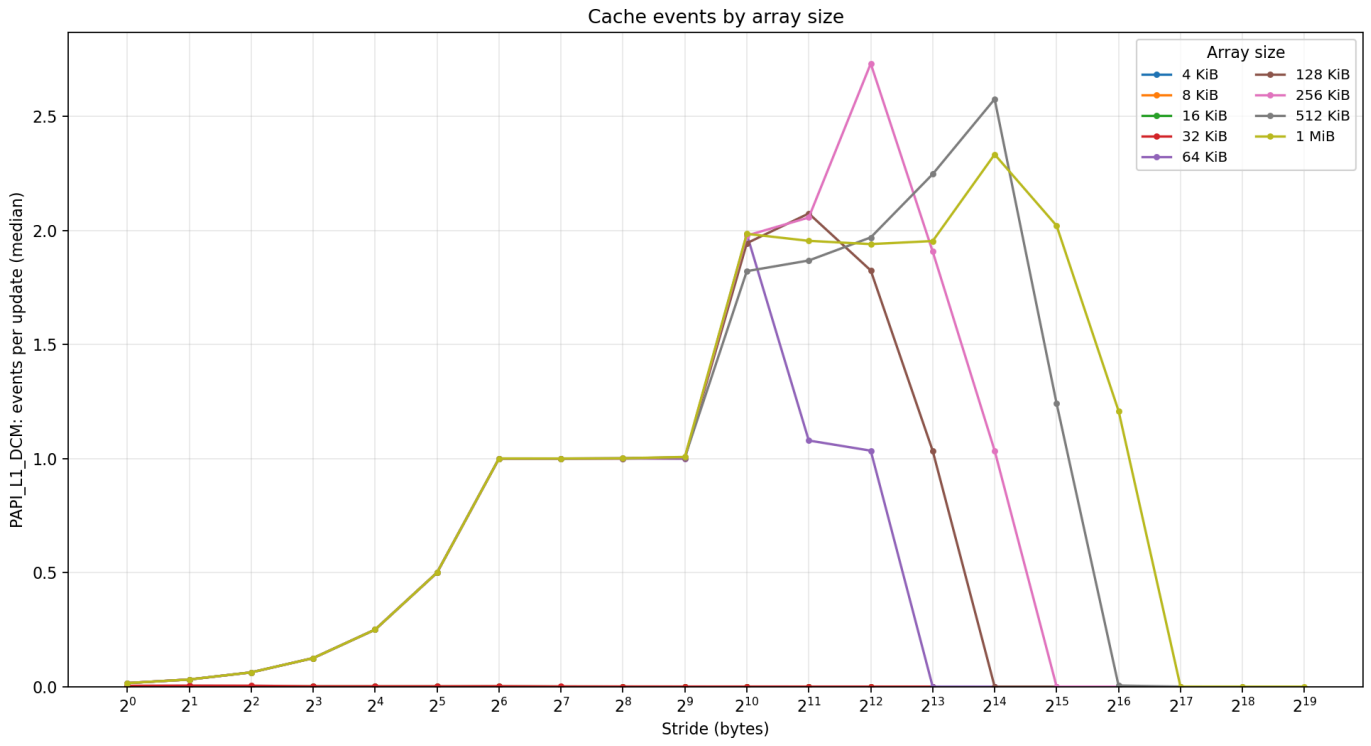
L1 Event Used: `PAPI_L1_DCM`

- **Short Description:** L1D cache misses
- **Native Event Description:** Number of cache lines brought into the L1D cache (L1D:REPLACEMENT)

L2 Event Used: `PAPI_L2_DCM`

- **Short Description:** L2D cache misses
- **Native Event Description:** Core-originated cacheable demand requests to L3 (LAST_LEVEL_CACHE_REFERENCES)

Question b) L1 Data-Cache Capacity Estimate

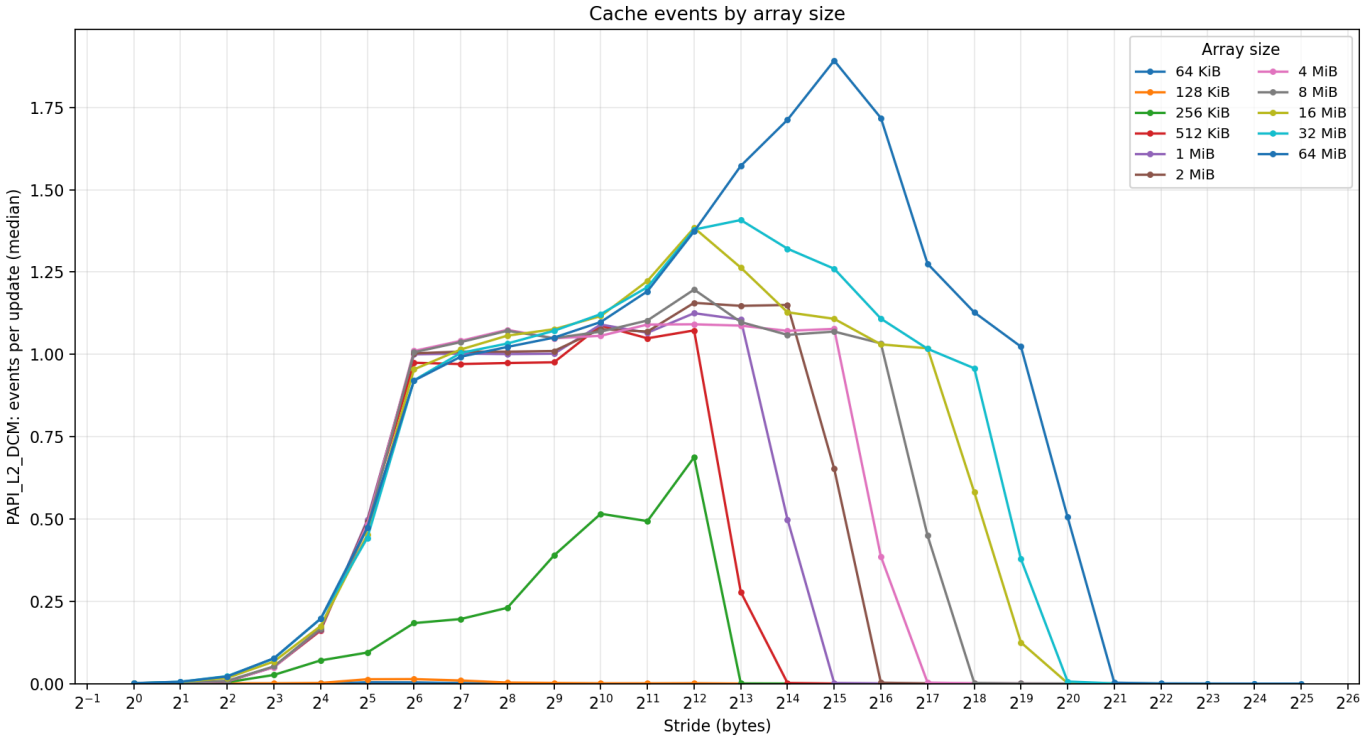


- **Estimated L1 Capacity: 32 KiB**
- **Array-size range used:** 4 KiB to 1 MiB

Explanation: The hardware counter data demonstrates a precise capacity boundary. For working sets up to 32 KiB, the L1 miss ratio is practically zero (e.g., ~0.002 misses per update at a 64-byte stride), indicating the array fits entirely within the cache. When the working set size increases to 64 KiB, the miss ratio abruptly jumps to exactly 1.0 at the same 64-byte stride. This step-function increase proves that the

64 KiB array exceeds the L1 capacity, causing every cache line accessed during an update pass to result in a capacity miss.

Question c) L2 Data-Cache Capacity Estimate



- **Estimated L2 Capacity: 256 KiB** (interval [256 KiB, 512 KiB])
- **Array-size range used:** 64 KiB to 64 MiB

Explanation: The hardware counter data for `PAPI_L2_DCM` clearly illustrates the L2 cache capacity boundary. For working sets of 64 KiB, 128 KiB, and 256 KiB, the L2 miss ratio remains low (e.g., approximately 0.18 misses per update at a 64-byte stride for the 256 KiB array). This indicates that while these arrays exceed the L1 capacity, they still fit comfortably within the L2 cache. However, when the working set size doubles to 512 KiB, the L2 miss ratio spikes to approximately 0.97 misses per update at the same 64-byte stride. This dramatic, near-complete miss rate proves that array sizes exceeding 256 KiB can no longer fit in the L2 cache, forcing the processor to evict data and fetch it from the slower L3 cache or main memory.

Question d) L1 Data-Cache Line Size Estimate

- **Selected Array Size: 128 KiB** *(Chosen because it is strictly larger than the 32 KiB L1 capacity, but smaller than the 256 KiB L2 capacity)*.
- **Estimated L1 Cache Line Size: 64 bytes**

Explanation: To estimate the cache line size, we analyze the relationship between the access stride and spatial locality. The data for the 128 KiB array demonstrates that the L1 miss ratio strictly scales as the stride doubles: at a 16-byte stride the median miss ratio is ≈ 0.25, and at a 32-byte stride it is ≈ 0.50. Once the stride reaches 64 bytes, the miss ratio reaches exactly ≈ 1.0 and plateaus at that value for larger strides (128 bytes, 256 bytes). This plateau indicates that spatial reuse within a fetched memory block has been fully exhausted at a 64-byte offset, confirming that the hardware fetches data in exact 64-byte cache lines.

Question e) Justification & Comparison

Capacity and Line Size Justifications:

- **L1 Capacity (32 KiB):** For arrays up to 32 KiB, the L1 miss ratio remains near zero (e.g., ~0.002 misses per update at a 64 B stride). When the array doubles to 64 KiB, the miss ratio jumps abruptly to 1.0 at the same 64 B stride, proving the data no longer fits in the L1 cache.
- **L2 Capacity (256 KiB):** For arrays up to 256 KiB, the L2 miss ratio remains low (e.g., ~0.18 at a 64 B stride). At 512 KiB, it spikes to ~0.97 misses per update at the same stride, confirming the 256 KiB L2 boundary.
- **L1 Line Size (64 bytes):** Using the 128 KiB array, the L1 miss ratio doubles as the stride doubles: ~0.25 at 16 B, ~0.50 at 32 B, and reaches exactly 1.0 at a 64 B stride. The miss ratio plateaus at 1.0 for all larger strides (128 B, 256 B), proving spatial reuse is exhausted at 64 bytes.

Comparison with Section 2: Both experiments were conducted on the same Intel Xeon E5-2650 machine (lab computer via ssh). The conclusions from both sections are in perfect agreement: L1 capacity is 32 KiB, L2 capacity is 256 KiB, and the cache line size is 64 bytes.

However, the hardware-counter measurements made the transitions significantly clearer. The timing measurements in Section 2 produced curves with gradual slopes due to the overlapping latency of CPU pipelining, out of order execution, loop overhead, and other noise. In contrast, the hardware counters measured the exact physical cache evictions, producing sharp step function graphs that provided mathematical certainty (e.g., jumping exactly from 0 to 1.0 misses per update) without timing ambiguity. There were no disagreements between the two methodologies.